TASK 6: Design the Control Structures

Name: Nagineni Sunil Kumar

Register Number: 192472330

1. Aim

To use if-else, switch, loops, and integer arithmetic to calculate fares and validate metro journeys.

2. Pseudocode

1. Start.
2. Read the zone and peak-hour status.
3. Use switch to select the base fare in paise.
4. If the zone is invalid, reject the journey.
5. If it is peak time, add the 20% surcharge using integer arithmetic.
6. Check whether the balance after payment will be at least 5000 paise.
7. If yes, deduct the fare and display the fare and balance.
8. Otherwise, display a low-balance message.
9. Repeat for all journeys.
10. Stop.

3. Completed C++ Program

```
#include <iomanip>

#include <iostream>

using namespace std;

enum class Zone { Inner, Middle, Outer };
```

```
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak) {
    long long base;
    switch (z) {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;
        case Zone::Middle:
            base = BASE_PAISE[1];
            break;
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;
        default:
            return -1;  }
    if (peak)
        base = base * (100 + PEAK_SURCHARGE_PERCENT) / 100;
    return base;}
bool canTravel(long long balance, long long f) {
```

```
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;}
int main() {
    long long balance = 10000;
    struct Trip {
        Zone zone;
        bool peak;   };
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}    };
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            cout << "Fare Rs " << f / 100 << '.'
                 << setw(2) << setfill('0') << f % 100
                 << setfill(' ');
        } else {
            cout << "Denied (low balance)";   }
        cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

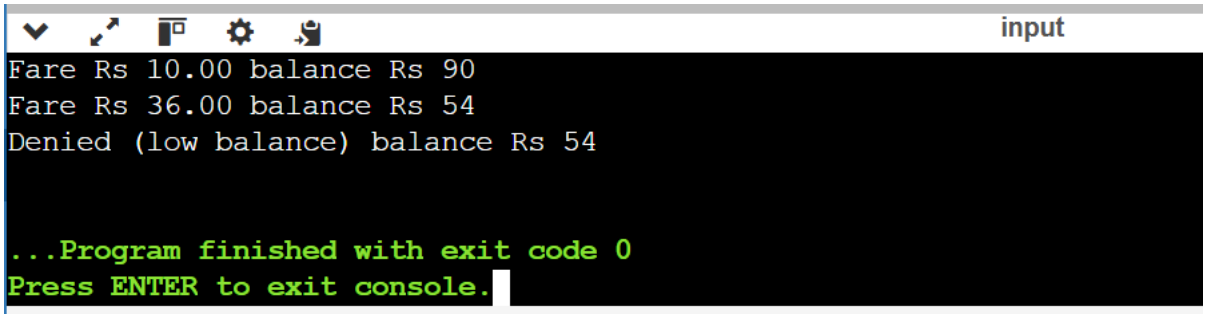
4. Expected Output

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Obtained output:



5. Peak-Hour Boundary Test

The correct peak check is:

(hour >= 8 && hour < 10) ||

(hour >= 17 && hour < 20)

Hour	**Peak?**
08:00	Yes
10:00	No
17:00	Yes
20:00	No

6. Control Structure Selection

- switch: Best for selecting a fare by zone.
- if-else: Best for peak-hour and balance decisions.

- Conditional operator ?:: Useful for simple two-way choices.
- for loop: Suitable when the number of journeys is known.
- Range-based for: Convenient for processing an array of journeys.
- while loop: Useful when journeys continue until a stop condition is reached.
- break: Exits a switch or loop.
- continue: Skips the current loop iteration.

7. Complexity

Each fare calculation takes O(1) time. Processing (n) journeys takes O(n) time. The fare function uses O(1) extra space.

8. Conclusion

Using switch for zone selection and if-else for validation makes the program readable. Integer paise keeps fare calculations consistent, and the minimum-balance check prevents invalid deductions.